

Stock Reconciliation — Design Document

System: SnackMaster ERP (React + Firebase Firestore + Express backend) **Scope:** Warehouse stock, machine stock, refill kits, and system data integrity.

1. Why Reconciliation Is Needed

Today the system tracks stock in **two independent places**, and they can drift apart:

Source	Where	Used By
<code>products.warehouseStock</code> (running counter)	<code>products/{id}</code> — updated via <code>increment()</code> on every Inward/Outward/Return	Inward, Outward, Returns pages (stock validation, Quick Stock View)
<code>warehouse_movements</code> (ledger)	One document per movement (INWARD, OUTWARD_KIT, OUTWARD_MANUAL, RETURN, EXPIRED_DAMAGED)	Warehouse Dashboard "Available Stock" (<code>inward - outward + returned - expired</code>), Ledger, PDF exports

Nothing guarantees these two agree. Drift happens when:

- A movement doc is written but the counter update fails (or vice versa) — the current pages do two writes that are not always atomic.
- Writes come from the client (browser), so a stale page can oversell stock (`warehouseStock` check happens before, not during, the write).
- Backend flows (kit dispatch, refill returns in `backend/index.js`) and frontend flows both mutate stock with slightly different field conventions.
- Data is manually edited in the Firestore console.
- Physical reality diverges from books: theft, damage, miscounts, refiller loading fewer units than issued.

There is currently **no adjustment mechanism, no physical count workflow, and no period close** — so once numbers drift, there is no controlled way to correct them.

2. Guiding Principles

1. **The ledger is the single source of truth.** `warehouse_movements` is the book of record. `products.warehouseStock` is only a cache/denormalized counter derived from it.
2. **Never edit balances directly.** Every correction is a new, immutable **ADJUSTMENT movement** with a reason, evidence, and an approver. No deleting or editing past movements.
3. **Reconcile at boundaries.** Stock changes hands at three boundaries — supplier→warehouse, warehouse→refiller, refiller→machine. Each boundary needs its own reconciliation.
4. **Count what you can see.** Book stock is only trustworthy if periodically compared against a physical count (stock take / cycle count).

5. **Close periods.** Snapshot balances daily/monthly so reconciliation compares small windows instead of re-summing the entire ledger forever, and locked periods cannot be silently changed.

3. The Four Reconciliation Types

3.1 System Reconciliation (Ledger vs Counter) — automated

Detects internal data drift. For each product:

```
ledgerBalance = Σ INWARD + Σ RETURN - Σ OUTWARD_* - Σ EXPIRED_DAMAGED
counterBalance = products.warehouseStock
variance       = counterBalance - ledgerBalance
```

- Run nightly (scheduled Cloud Function / cron on the Express backend).
- Write results to a **reconciliation_runs** doc; alert admin when any variance $\neq 0$.
- Resolution: the ledger wins. Reset **warehouseStock** = **ledgerBalance** via a logged repair action (not a movement — this is fixing the cache, not the books).

3.2 Warehouse Reconciliation (Book vs Physical) — stock take

Detects real-world shrinkage. Workflow:

1. Admin starts a **Stock Take** for a set of products (full count or cycle count of a few SKUs per week).
2. System freezes the book quantity per product at start (**bookQty**).
3. Counter enters physical quantities (**countedQty**), blind — do not show **bookQty** while counting.
4. System computes **variance** = **countedQty** - **bookQty** per line.
5. A **second person** (approver) reviews variances; approved variances generate **ADJUSTMENT** movements (positive or negative) with **reason** (e.g. DAMAGE, THEFT, COUNT_ERROR, DATA_ENTRY_ERROR) and **stockTakeId** reference.
6. Stock take is closed and locked.

3.3 Refiller / Kit Reconciliation (Issued vs Loaded vs Returned)

The warehouse→refiller→machine chain must always balance per kit/outward issue:

```
issuedQty = loadedIntoMachineQty + returnedQty + varianceQty
```

- When a kit is completed (refiller confirms loading), the refiller records per product: loaded qty and returned qty.
- **varianceQty** $\neq 0$ creates a **pending variance** attributed to the refiller (**issuedTo**), requiring admin resolution (accept as loss → **ADJUSTMENT**, or correct the entries).
- The Manual Outward flow (one refiller, multiple product×machine rows) already writes one movement per combination — this gives the per-machine granularity needed; reconciliation groups them by **issuedTo** + date.

3.4 Machine Reconciliation (Expected vs Actual at Visit)

Per machine, between two refill visits:

```
expectedClosing = openingCount + refilledQty - vendedQty(sales transactions)
variance        = actualCountAtVisit - expectedClosing
```

- On every refill visit, the refiller first records the **actual count per slot** (this is already close to the slot `current_qty` flow) before loading.
- Variance indicates vend failures, theft, or unrecorded sales — logged per machine per visit, trended over time to spot problem machines/refillers.

4. Data Model

4.1 New movement type: `ADJUSTMENT`

Added to `warehouse_movements` (keeps the single-ledger principle):

```
{
  type: "ADJUSTMENT",
  direction: "IN" | "OUT",           // sign of the correction
  quantity: 5,
  productId, productName, orgId,
  reason: "STOCK_TAKE" | "DAMAGE" | "THEFT" | "COUNT_ERROR" | "DATA_ENTRY_ERROR" |
  "SYSTEM_REPAIR",
  referenceId: "<stockTakeId | reconciliationRunId | kitId>",
  remarks: "...",
  requestedBy: "<email>",
  approvedBy: "<email>",           // must differ from requestedBy
  performedBy: "<email>",
  movementDate, createdAt
}
```

Dashboard math becomes: `available = inward - outward + returned - expired ± adjustments`.

4.2 `stock_takes` collection

```
stock_takes/{id} = {
  orgId, status: "OPEN" | "PENDING_APPROVAL" | "CLOSED",
  scope: "FULL" | "CYCLE",
  startedBy, startedAt, approvedBy, closedAt,
  lines: [{
    productId, sku, productName,
    bookQty,           // frozen at start
    countedQty,        // entered by counter
    variance,          // countedQty - bookQty
  ]
}
```

```

    resolution: "ADJUSTED" | "RECOUNTED" | "IGNORED",
    adjustmentMovementId
  }]
}
```

4.3 `stock_snapshots` collection (period close)

```

stock_snapshots/{orgId_YYYY-MM-DD} = {
  orgId, snapshotDate, createdAt, locked: true,
  products: {
    [productId]: { sku, opening, inward, outward, returned, expired, adjustments,
closing }
  }
}
```

- Generated nightly by the backend.
- `closing` of day N must equal `opening` of day N+1 — a cheap built-in integrity check.
- Reports and dashboards read snapshots + today's movements instead of scanning the full ledger (fixes the growing-forever query in `WarehouseDashboard.jsx`).

4.4 `reconciliation_runs` collection

```

reconciliation_runs/{id} = {
  orgId, runDate, type: "SYSTEM" | "MACHINE" | "KIT",
  status: "CLEAN" | "VARIANCES_FOUND" | "RESOLVED",
  lines: [{ productId | machineId | kitId, expected, actual, variance, resolvedBy,
resolution }]
}
```

5. Process & Controls

Control	Rule
Segregation of duties	The person who counts is not the person who approves adjustments (<code>requestedBy</code> $\neq$ <code>approvedBy</code>).
Blind counts	Counter never sees book quantity while counting.
Immutability	Movements and closed stock takes/snapshots are never edited or deleted; corrections are new ADJUSTMENT movements. Enforce with Firestore security rules (<code>allow update, delete: if false</code> on <code>warehouse_movements</code>).
Thresholds	Auto-approve variances below a configurable value (e.g. ≤ 2 units or $\leq ₹100$); larger variances require explicit admin approval.

Control	Rule
Cadence	System reconciliation: nightly. Cycle counts: weekly (fast movers) / monthly (slow movers). Full stock take: quarterly. Machine reconciliation: every refill visit. Period close: nightly snapshot, monthly lock.
Attribution	Every kit/outward variance is attributed to issuedTo (refiller) so shrinkage patterns per person/machine are visible.

6. Server-Side Enforcement (prevents drift at the source)

Reconciliation gets much cheaper if drift stops being created. Move stock mutations from the browser to the Express backend (or Cloud Functions), wrapped in Firestore **transactions**:

```
POST /warehouse/movements → runTransaction:
  1. re-read products/{id}.warehouseStock
  2. reject if outward qty > current stock (no more stale-page overselling)
  3. write movement doc + counter increment atomically
```

Benefits:

- Movement + counter always succeed or fail together (today's **writeBatch** in the Outward page is atomic, but the Inward/Returns pages still do two separate writes, and none re-validate stock inside a transaction).
- One code path for frontend and backend flows → consistent field conventions (**issuedBy**, **issuedTo**, **destination**, **performedBy**).
- Security rules can then deny direct client writes to **warehouse_movements** and **products.warehouseStock** entirely.

7. Implementation Roadmap

Phase 1 — Visibility (no behavior change)

1. Nightly system reconciliation job (ledger vs counter) + admin report page showing per-product variance.
2. "Repair counter" action (sets **warehouseStock** = **ledgerBalance**, logged to **admin_actions**).

Phase 2 — Corrections 3. **ADJUSTMENT** movement type + admin UI (reason codes, dual approval). 4. Include adjustments in Dashboard/Ledger math and exports.

Phase 3 — Physical counts 5. Stock take module (create, blind count entry, variance review, auto-generate adjustments). 6. Cycle count scheduling (ABC classification: fast movers counted more often).

Phase 4 — Boundary reconciliation 7. Kit close-out screen: loaded vs returned vs issued per product; variance attribution to refiller. 8. Machine visit reconciliation: actual slot counts vs expected (opening + refill – sales).

Phase 5 — Hardening 9. Move all stock mutations behind backend transactions; lock down Firestore rules.
10. Nightly `stock_snapshots` + monthly period lock; dashboards read snapshots instead of full ledger scans.

8. Success Metrics

- System variance (ledger vs counter): 0 products with non-zero variance on nightly run.
- Book-vs-physical accuracy: > 98% of SKUs within tolerance at cycle counts.
- Kit variance rate: < 1% of issued units unaccounted per month.
- Time to resolve a flagged variance: < 48 hours.
- 100% of adjustments carry reason + approver.